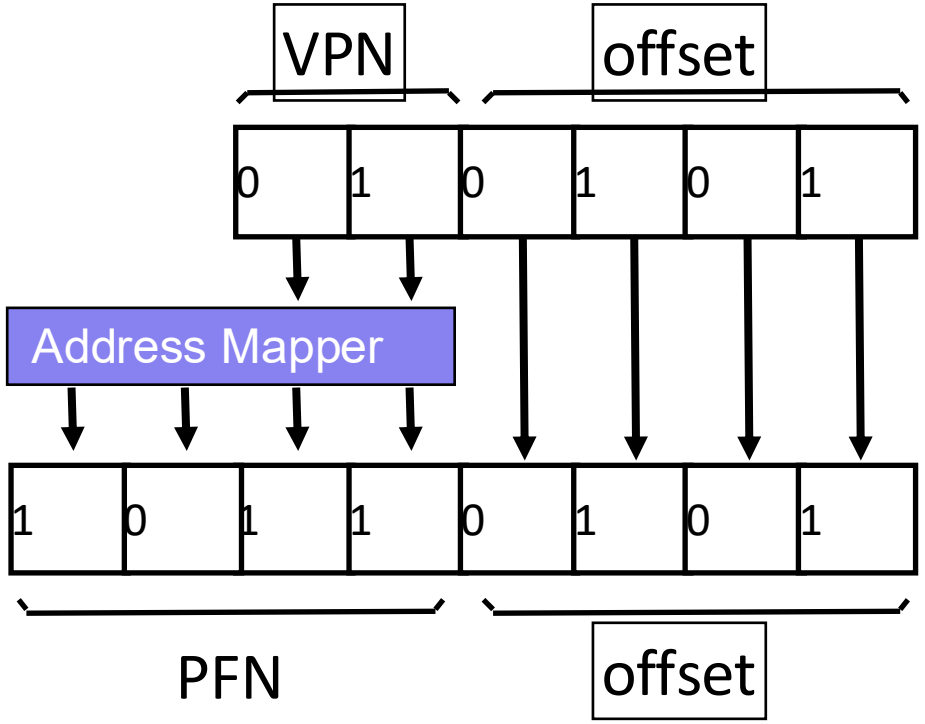
Memory Virtualization



Review: Virt ➔ Phys page mapping

Number of bits in the virtual address not equal the number of bits in the physical address



How should OS translate VPN to PFN?

For segmentation, OS used a formula (e.g., phys addr = virt_offset + base_reg)

For paging, OS needs a different mapping mechanism

Q: What data structure is good?

Linear Page Table

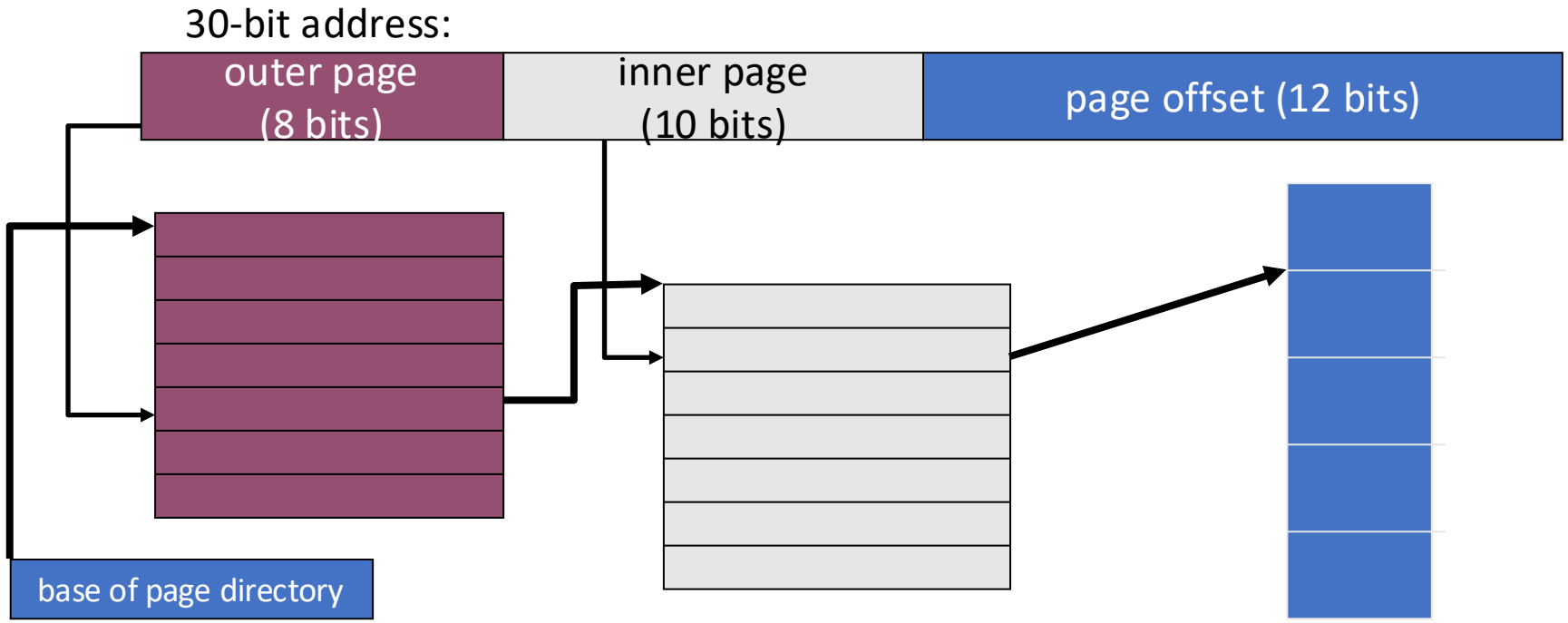
A simple option: an array

Review: Multilevel Page Tables

Goal: Allow page tables to be allocated non-contiguously

Idea: Page the page tables

- Creates multiple levels of page tables; outer level page directory
- Only allocate page tables for pages in use
- hardware can walk known structure; e.g. x86

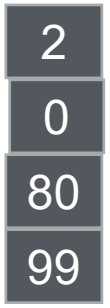


Review: Memory Accesses with Pages

```
0x0010:  movl   0x1100, %edi
0x0013:  addl $0x3, %edi
0x0019:  movl   %edi, 0x1100
```

Assume 14-bit virtual address
Assume PTE's are 4 bytes
Assume 4KB pages
How many bits for offset?
Assume a 3-level page table

Conceptual linear view of page table



Earlier: How many mem refs with linear page table?

10 (3 inst + 2 load/store + 5 page table)

Physical Memory Accesses with Paging?

1) Fetch instruction at virtual addr 0x0010; vpn?

- Access page table to get pfn for vpn 0
- Mem ref 1,2,3: get pfn
- Learn vpn 0 is at pfn 2
- Fetch instruction at 0x2010 (Mem ref 4)

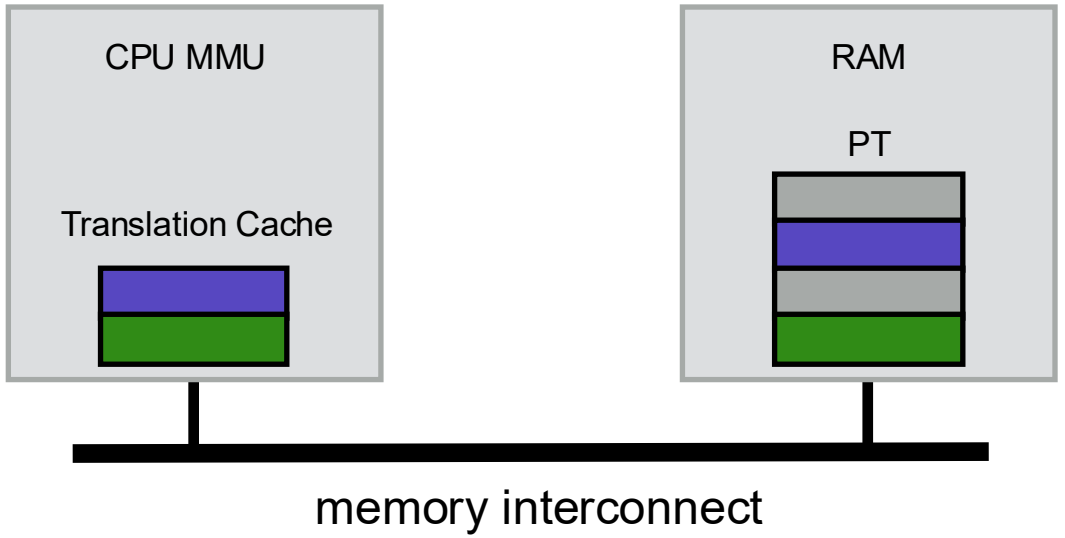
Exec, load from virtual addr 0x1100; vpn?

- Access page table to get pfn for vpn 1
- Mem ref 5,6,7: get pfn
- Learn vpn 1 is at pfn 0
- Movl from 0x0100 into reg (Mem ref 8)

Use of a k-level page table results in (k+1) times more memory refs

Review: TLBs cache page translations

We couldn't store entire page table in MMU, but we can store a fast cache



Some popular entries

TLB: **T**ranslation **L**ookaside **B**uffer

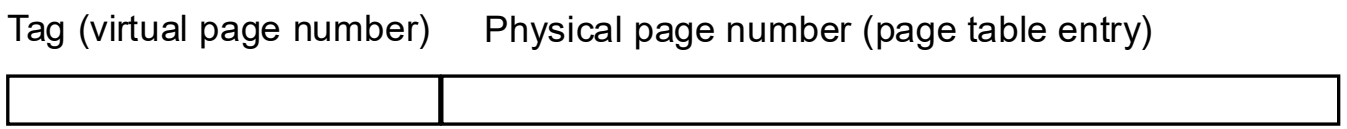
Review: TLB Control Flow

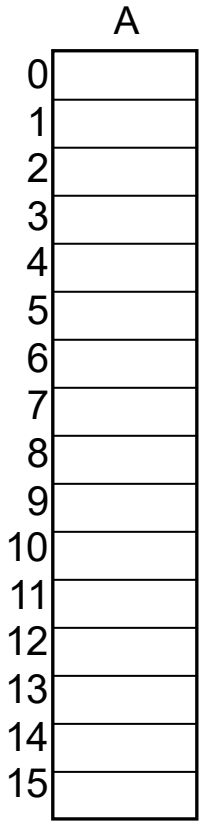
```
1   VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2   (Success, TlbEntry) = TLB_Lookup(VPN)
3   if (Success == True)   // TLB Hit
4       if (CanAccess(TlbEntry.ProtectBits) == True)
5           Offset   = VirtualAddress & OFFSET_MASK
6           PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7           Register = AccessMemory(PhysAddr)
8       else
9           RaiseException(PROTECTION_FAULT)
10  else                    // TLB Miss
11      PTEAddr = PTBR + (VPN * sizeof(PTE))  // assume simple linear page table
12      PTE = AccessMemory(PTEAddr)
13      if (PTE.Valid == False)
14          RaiseException(SEGMENTATION_FAULT)
15      else if (CanAccess(PTE.ProtectBits) == False)
16          RaiseException(PROTECTION_FAULT)
17      else
18          TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19          RetryInstruction()
```

Hardware-managed TLB

TLB Organization

TLB Entry





Lookup

- Calculate set (tag % num_sets)
- Search for tag within resulting set

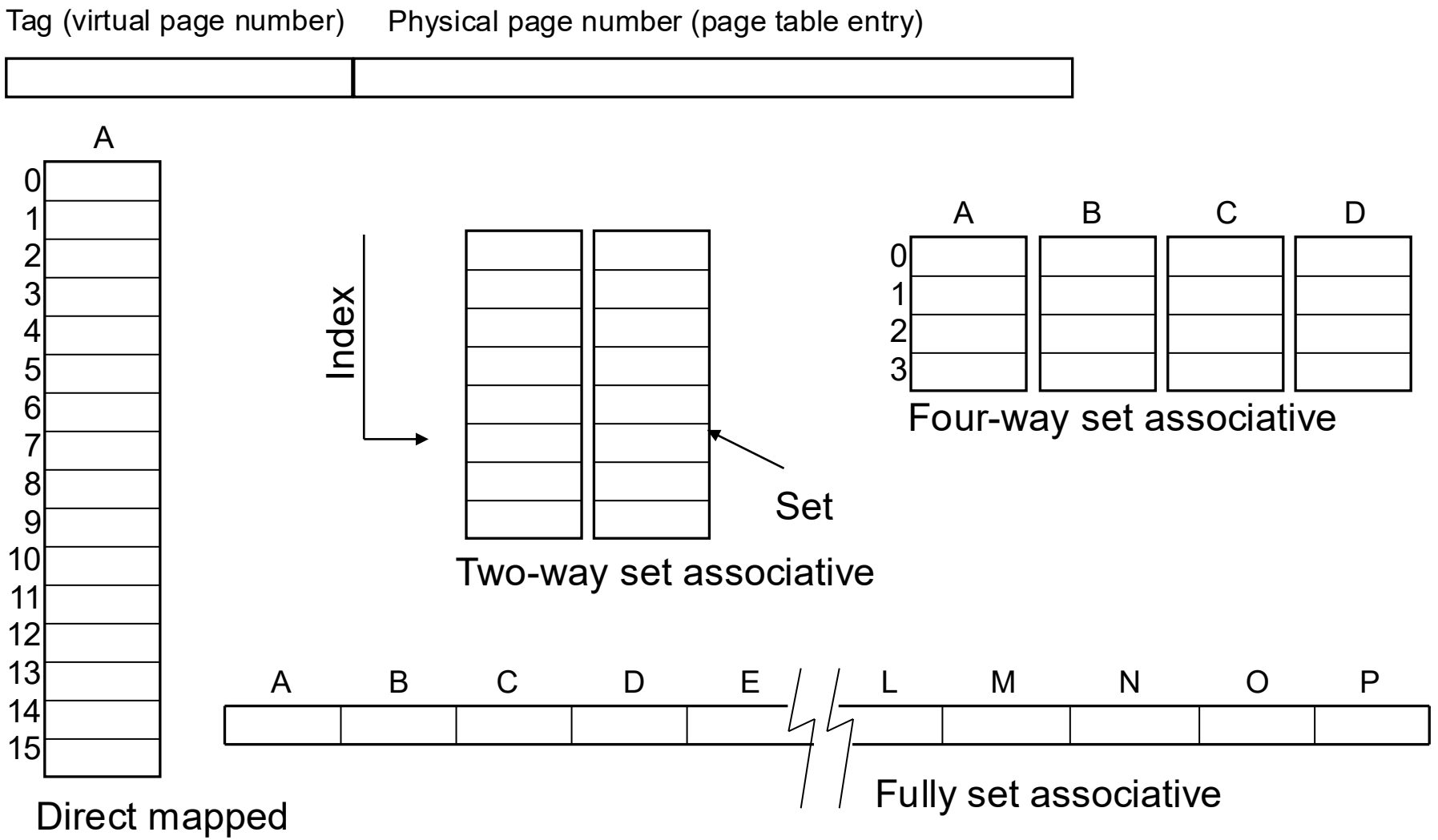
Where is VPN (tag) 18 located?

2

Direct mapped (num sets = 16)

TLB Organization

TLB Entry



TLB Associativity Trade-offs

Higher associativity

+ Better utilization, fewer collisions
– Slower
– More (expensive) hardware

Lower associativity

+ Fast
+ Simple, less hardware
– Greater chance of collisions

Modern architectures: TLBs in a hierarchy; use a variety of associativity levels

Examples in the subsequent slides assume full associativity.

Array Iterator (with TLB)

```
int sum = 0;
for (i = 0; i < 2048; i++){
     sum += a[i];
}
```

Assume following virtual address stream:
load 0x1000

load 0x1004

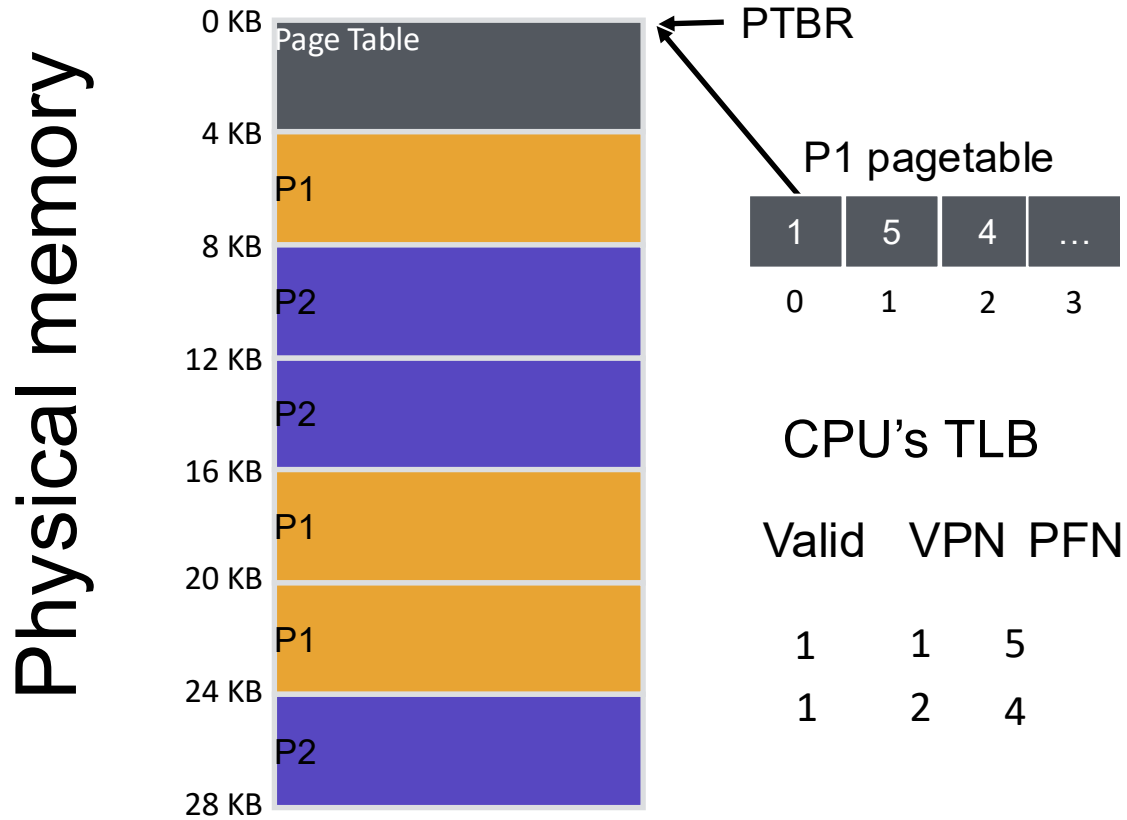
load 0x1008

load 0x100C
...

What will TLB behavior look like?

TLB Accesses: Sequential Example

```
sizeof(PTE) == 4
```



Virtual	Physical
load 0x1000	load 0x0004
	load 0x5000
load 0x1004	(TLB hit)
	load 0x5004
load 0x1008	(TLB hit)
load 0x100c	load 0x5008
	(TLB hit)
	load 0x500C
…	…
load 0x2000	load 0x0008
load 0x2004	load 0x4000
	(TLB hit)
	load 0x4004

4KB page size

Performance Of TLB?

```
int sum = 0;
for (i=0; i<2048; i++) {
      sum += a[i];
}
```

Calculate miss rate of TLB for a[i]:
TLB misses / # TLB lookups

TLB lookups?
= number of accesses to a = 2048

TLB misses?
= number of unique pages accessed
= 2048 / (elements of ‘a’ per 4K page)
= 2K / (4K / sizeof(int)) = 2K / 1K = 2

Miss rate?
2/2048 = 0.1%

Hit rate? (1 – miss rate)
99.9%

Would hit rate get better or worse with smaller pages?
Worse

TLB

How can system improve TLB performance (hit rate) given fixed # TLB entries available in hardware?

Increase page size

- Fewer unique page translations needed to access same amount of memory

The TLB's reach in terms of physical memory size:

- Number of TLB entries * Page Size

"Huge pages" used in many real systems.
Optionally add to your application (e.g. using mmap)

TLB Performance with Workloads

Sequential array accesses almost always hit in TLB

- Access goes directly to desired memory address; Fast

What access pattern will be slow?

- Highly random, with little/no repeat accesses

Caches rely on
spatial and temporal locality.

Workload Access Patterns

Workload A

```
int sum = 0;
for (i=0; i<2048; i++) {
      sum += a[i];
}
```

Workload B

```
int sum = 0;
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
```

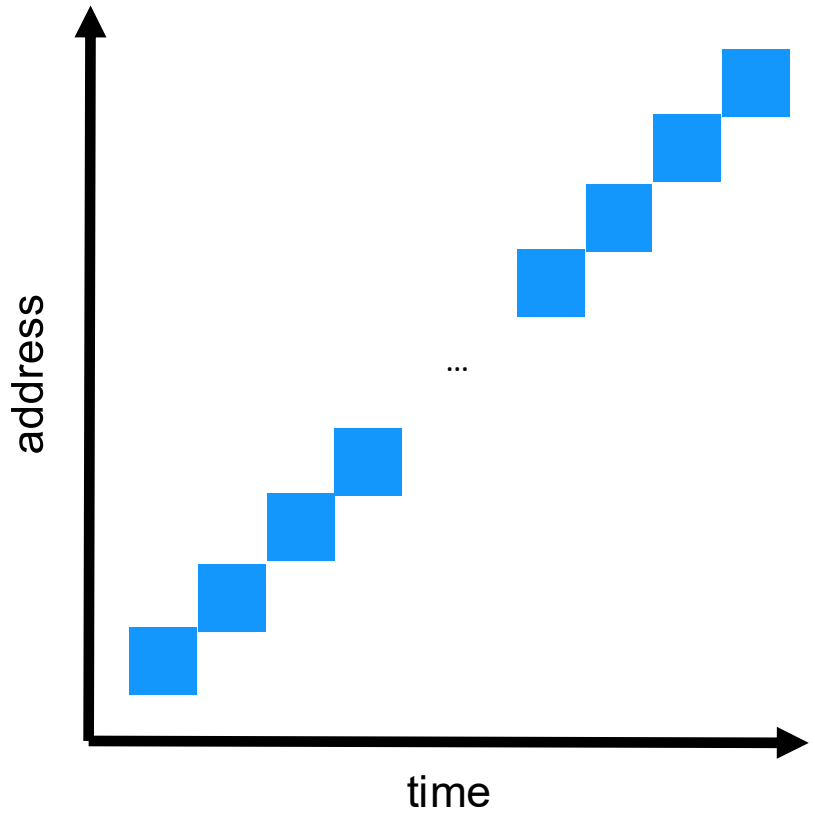
Workload Access Patterns

Workload A

```
int sum = 0;
for (i=0; i<2048; i++) {
      sum += a[i];
}
```

Spatial Locality

Sequential Accesses



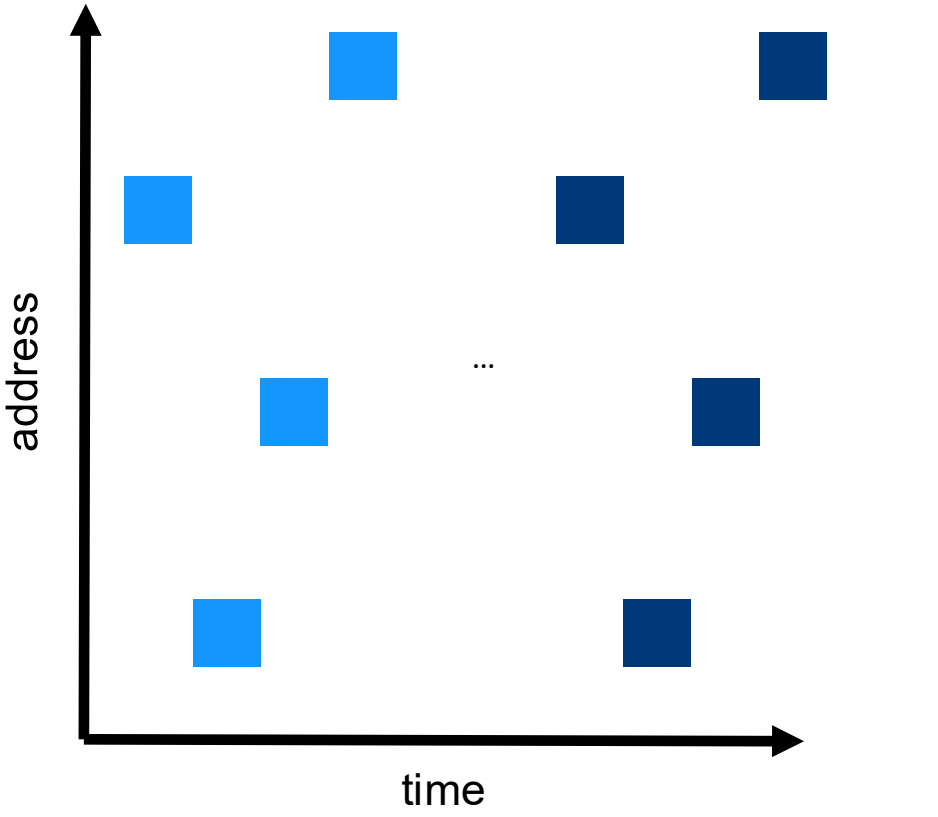
Workload Access Patterns

Workload B

```
int sum = 0;
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
```

Temporal Locality

Repeated Accesses to the same entity



Workload Locality

Spatial Locality: future access will be to nearby addresses

Temporal Locality: future access will be repeats to the same address

What TLB characteristics are essential to facilitate each type?

Spatial:

- Access same page repeatedly; need same VPN → PFN translation
- Same TLB entry re-used

Temporal:

- Access the same address in the near future
- Same TLB entry re-used in near future
- What happens if the TLB entries are fully used up?
- Eviction! How near is the access in the future? How many TLB entries are there? What's the TLB eviction policy?